

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра технической кибернетики

ОТЧЁТ

по лабораторным работам

по дисциплине «Технологии программирования на Python»

Выполнил: Балакирев Илья Олегович,
Группа: 6302-010302D,
Преподаватель: Мотз Георгий Александрович.

Самара 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1.1 Цель работы	3
1.2 Требования к проекту.....	4
2 Основная часть	8
2.1 Архитектура проекта	8
2.2 Внутренняя реализация	9
2.3 Инструкция по установке и запуску	10
ЗАКЛЮЧЕНИЕ	14
ПРИЛОЖЕНИЕ А Ссылки на репозиторий и пакет	15
ПРИЛОЖЕНИЕ Б Список зависимостей	16
ПРИЛОЖЕНИЕ В Код конфигурации логгера	17

ВВЕДЕНИЕ

1.1 Цель работы

Основной целью лабораторных работ в рамках дисциплины «Технологии программирования на Python» являлось создание комплексного, масштабируемого и хорошо структурированного приложения на языке Python, предназначенного для взаимодействия с открытыми данными коллекции Метрополитен музея (The Metropolitan Museum of Art). В процессе выполнения пяти лабораторных работ ставилась задача не только разработать функциональное приложение `metetl`, но и углубленно освоить спектр технологий программирования на Python.

К ним относятся:

- Объектно-ориентированное программирование (ООП) – стиль программирования на основе описания типов/моделей предметной области и их взаимодействия, представленных порождением из прототипов или как экземпляры классов, которые образуют иерархию наследования;
- Обработка и анализ больших объемов данных с помощью библиотеки `pandas` (для структурированных табличных данных, используется в аналитике, машинном обучении, и т.д.);
- Эффективная визуализация данных с использованием `matplotlib`;
- Асинхронное программирование для ввода-вывода с использованием `aiohttp` и `aiofiles` – концепция программирования, которая заключается в том, что результат выполнения функции доступен не сразу, а через некоторое время в виде некоторого асинхронного (нарушающего обычный порядок выполнения) вызова;
- Параллельные вычисления с помощью `multiprocessing` – способ организации компьютерных вычислений, при котором программы разрабатываются как набор взаимодействующих вычислительных процессов, работающих параллельно (одновременно);

- Логирование для мониторинга и отладки – процесс записи и хранения информации о событиях, действиях и состояниях системы, приложений или пользователей;
- Разработка удобного интерфейса командной строки (CLI) с помощью `argparse` (стандартная библиотека Python, предназначенная для обработки аргументов командной строки);
- Полный цикл создания и сборки готового к распространению Python-пакета.

1.2 Требования к проекту

Проект разрабатывался в строгом соответствии с заданиями, представленными в лабораторных работах.

Требования лабораторной работы №1 включали реализацию механизма взаимодействия с публичным API Метрополитен-музея для получения списка объектов и скачивания случайного изображения (классификация `Paintings`), используя данные из CSV-файла `MetObjects.csv`, и алгоритмов обработки изображений (`rgb_to_grayscale` – приведение к полутоновому, `convolution` – свёртка с двумерной маской, `gaussian_blur` – гауссово размытие, `sobel_edge_detection` – выделение границ оператором Собеля, `gamma_correction` – гамма-коррекция, `histogram_equalization` – выравнивание гистограммы) с использованием библиотек `numpy` и `opencv`. Необходимо было сравнить работу собственных алгоритмов с библиотечными функциями `cv2`.

Для лабораторной работы №2 основным требованием стал переход от процедурной к объектно-ориентированной архитектуре путём проектирования абстрактного класса `Artwork`, определяющий общий паттерн поведения для всех типов изображений и реализующий операции, независимые относительно цветового пространства. С целью экономии памяти и ограничения доступа используется механизм `__slots__ = ('_image', '_metadata', '_processed_images')`. Доступ к ним осуществляется при помощи геттеров, которые обеспечивают инкапсуляцию. Класс предоставляет методы для

обработки изображений аналогично также, как и в лабораторной работе №1. Создан класс `Metadata` для хранения информации о картине (`object_id`, `title`, `artist`, `date`, `medium`, `dimensions`).

Для замера времени выполнения функций был реализован декоратор `timer_decorator`, который выводит в консоль время работы декорируемой функции.

В проекте присутствует наследование от `Artwork` создание конкретных реализаций: `GrayscaleArtwork` для полутоновых изображений (из двумерного массива (H, W)) и `ColorArtwork` для цветных изображений (трёхмерный массив (H, W, 3)) с перегрузкой конструкторов, добавлением необходимых методов и атрибутов. Проект также содержит класс `ImageProcessor`, который управляет процессом загрузки, обработки и сохранения изображений: он загружает изображение и метаданные из файла, выполняет последовательную обработку с замером времени и сохраняет результаты в указанную директорию. В будущем, этот класс будет использоваться для параллельной обработки изображений.

В лабораторной работы №3 требования заключались в создании скрипта для анализа большого CSV-файла на основе генераторного пайплайна без загрузки всего файла в оперативную память. Построение архитектуры на основе ленивых вычислений. Каждый этап обработки (`chunk_reader` – чтение чанками, `culture_filter` – фильтрация, `aggregate` – агрегация, `run_accumulator` – связывает все три генератора в цепочку и итерируется по ней) должен быть реализован в виде отдельной генераторной функции и вместе должны составлять единый конвейер последовательного вызова друг друга

```
run_accumulator(aggregate(culture_filter(chunk_reader(csv_path))))).
```

Для топ-10 самых часто повторяющихся культур должны вычисляться средний возраст объектов на момент поступления, 95% доверительный интервал для среднего и 95% интервал рассеяния. Визуализация полученных

результатов с помощью `matplotlib` в виде столбцовой диаграммы, отображающей средний возраст, доверительные интервалы и интервалы рассеяния для выбранных категорий, и временного графика изменения возраста объекта при поступлении со скользящим средним для культуры с самой длительной историей.

В лабораторной работе №4 было необходимо обеспечить поддержку массовой обработки изображений (количество задается через параметр командной строки).

Требовалось реализовать асинхронную загрузку изображений с использованием библиотек `aiohttp` и `aiofiles`, а также `asyncio.Semaphore` для управления количеством одновременных соединений к API музея.

В связи с тем, что в Python присутствует глобальная блокировка интерпретатора (GIL), которая ограничивает выполнение только одного потока в одном интерпретаторе, в свою очередь процессы работают независимо и не делят один общий GIL. Вычислительно сложные задачи свёртки необходимо было перевести в режим параллельного выполнения через `concurrent.futures.ProcessPoolExecutor` для задействования нескольких ядер CPU и соответственно ускорения. Также происходит добавление подробного логирования для отслеживания этапов выполнения асинхронных и параллельных задач и логируется время выполнения каждого этапа программы.

Для лабораторной работы №5 необходимо было провести интеграцию модуля `logging` для настройки логирования в файл (уровень `DEBUG` – самый подробный уровень) и краткого вывода в консоль (уровень `INFO`). Был создан файл `logging_config.json`, который представляет собой конфигурационный файл для модуля `logging` из стандартной библиотеки Python. Он содержит настройки, определяющие, как именно программа будет вести записи о своей работе. Этот файл позволяет гибко изменять параметры логирования без необходимости изменять исходный код программы. Для файла будет записываться в формате: время – название модуля – уровень – имя файла:строка –

сообщение. В свою же очередь в консоли будет: время – название модуля – уровень – сообщение. Это нужно для форматов, которые определяют как именно буду вестись логи. Также описываются обработчики, отвечающие за отправку записей в соответствующее место.

Разработан полноценный CLI-интерфейс с помощью `argparse`, включающего команды:

1. `metetl prepare --csv data/MetObjects.csv --output data/to_download.json` (подготовка JSON-файла с метаданными выбранных изображений),
2. `metetl process --input data/to_download.json --output images --num 5` (запуск пайплайна по скачиванию и параллельной обработке изображений),
3. `metetl analyze --csv data/MetObjects.csv --output-dir data/plots` (запуск анализа данных CSV-файла с построением графиков),
4. `metetl --help` (вывод справки о доступных командах и параметрах).

С более подробным результатом выполнения и логами можно ознакомиться в пункте 2.2.

Подготовлена структура пакета для распространения, написан конфигурационный файл `pyproject.toml` для управления зависимостями с минимальными версиями, точкой входа консольного приложения и директиву включения JSON-файла конфигурации логирования в состав пакета. Сами дистрибутивы `tar` и `wheel` формируется командой `python -m build`. Затем опубликовано в репозиторий TestPyPI при помощи утилиты `twine`.

2 Основная часть

2.1 Архитектура проекта

Проект структурирован как стандартный Python-пакет, разделенный на несколько логических модулей для обеспечения четкости и поддерживаемости кода. Основные модули и их функционал:

- `metetl.cli`: обеспечивает интерфейс командной строки. Содержит парсер аргументов (`argparse`) и функции-обработчики для каждой из доступных команд: `prepare` (подготовка JSON-файла с метаданными), `process` (запуск пайплайна скачивания и обработки), `analyze` (запуск пайплайна анализа данных);
- `metetl.images`: модуль для скачивания и обработки изображений.

Ключевые компоненты:

- `models`: содержит классы данных: `Metadata` (`@dataclass` для хранения метаданных), `Artwork` (Абстрактный базовый класс для представления произведения искусства), `ColorArtwork`, `GrayscaleArtwork` (конкретные реализации, наследующие от `Artwork` и реализующие методы обработки);
- `async_downloader`: реализует асинхронную загрузку изображений. Использует `aiohttp` для HTTP-запросов и `aiofiles` для записи на диск, а также `asyncio.Semaphore` для контроля количества одновременных загрузок;
- `parallel_processor`: реализует параллельную обработку загруженных изображений. Использует `concurrent.futures.ProcessPoolExecutor` для распределения вычислительной нагрузки между ядрами процессора;
- `metetl.analysis`: Модуль для анализа данных. Включает: `generators` (Содержит набор генераторных функций, формирующих пайплайн для обработки данных чанками), `analysis` (Содержит

функции для вычисления статистик), plotting (Содержит функции для визуализации результатов);

- `metetl.logging_config`: Модуль для настройки системы логирования. Загружает конфигурацию из JSON-файла, создает файловый (`logs/metetl.log` с уровнем `DEBUG`) и консольный (уровень `INFO`) обработчики;
- `metetl.decorators`: Содержит декораторы для замера времени выполнения (`timer_decorator`, `async_timer_decorator`), которые применяются к соответствующим методам в классах.

2.2 Внутренняя реализация

При разработке проекта были применены следующие технологии и принципы.

Объектно-ориентированное программирование (ООП). Проект построен на принципах ООП: инкапсуляция (данные изображения и метаданные скрыты внутри класса `Artwork`), наследование (классы `ColorArtwork` и `GrayscaleArtwork` наследуют от `Artwork`), полиморфизм (перегрузка методов обработки в наследниках) и абстракция (абстрактный базовый класс `Artwork` задает общий интерфейс для работы с любым типом изображений).

Использование `aiohttp` и `asyncio` позволило эффективно загружать несколько изображений одновременно, используя асинхронность, не блокируя основной поток выполнения. Это значительно ускорило процесс по сравнению с синхронным подходом.

Применение `ProcessPoolExecutor` позволило задействовать все доступные ядра процессора для выполнения вычислительно-емких операций обработки изображений (свертка, фильтрация и т.д.), что также привело к существенному ускорению работы программы. Это является параллельной обработкой изображения.

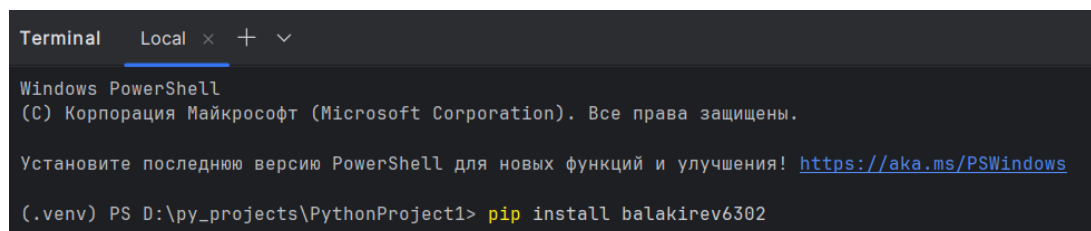
Анализ и визуализация данных. Библиотека `pandas` использовалась для чтения CSV-файла и манипуляции данными. Реализация пайплайна через генераторы (`chunk_reader`, `filter`, `aggregate`) обеспечила возможность обработки

файла произвольного размера, не загружая его целиком в оперативную память. А для визуализации статистики и выявления закономерностей в данных использовалась библиотека `matplotlib` (построение столбцовых диаграмм с доверительными интервалами и временных графиков со скользящим средним).

Также использовалось логирование и интерфейс командной строки (CLI). Внедрение модуля `logging` с конфигурацией через JSON позволило гибко настроить уровни, форматы и назначение (консоль/файл) логов, что критически важно для отладки и мониторинга. В свою очередь, использование `argparse` предоставило удобный способ взаимодействия с программой через терминал, позволяя выбирать конкретные действия (`prepare`, `process`, `analyze`) и задавать параметры (путь к файлам, количество изображений и т.д.).

2.3 Инструкция по установке и запуску

Для установки пакета и его зависимостей необходимо выполнить следующую команду в терминале:



```
Terminal Local x + v
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

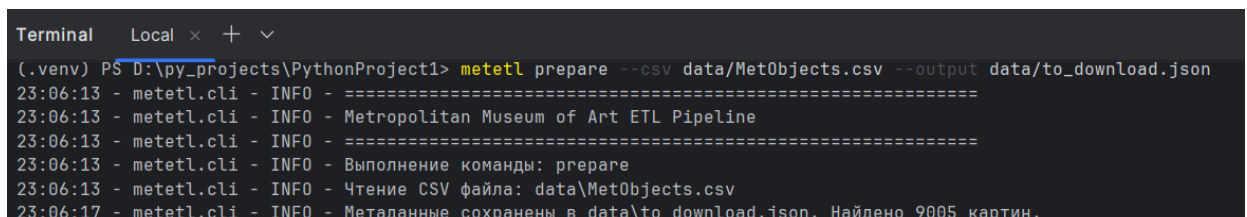
Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

(.venv) PS D:\py_projects\PythonProject1> pip install balakirev6302
```

Рисунок 1 – Установка приложения.

После успешной установки приложения и зависимостей, для запуска через интерфейс командной строки доступны три основные команды.

1. Подготовка JSON-файла с метаданными. Эта команда сканирует исходный CSV-файл, находит все объекты с классификацией "Paintings" и создает JSON-файл с их метаданными для дальнейшего использования.



```
Terminal Local x + v
(.venv) PS D:\py_projects\PythonProject1> metetl prepare --csv data\MetObjects.csv --output data\to_download.json
23:06:13 - metetl.cli - INFO - =====
23:06:13 - metetl.cli - INFO - Metropolitan Museum of Art ETL Pipeline
23:06:13 - metetl.cli - INFO - =====
23:06:13 - metetl.cli - INFO - Выполнение команды: prepare
23:06:13 - metetl.cli - INFO - Чтение CSV файла: data\MetObjects.csv
23:06:17 - metetl.cli - INFO - Метаданные сохранены в data\to_download.json. Найдено 9005 картин.
```

Рисунок 2 – Пример использования команды “prepare”.

```

1  [
2  {
3      "object_id": "35155",
4      "object_number": null,
5      "title": "Guidobaldo II della Rovere, Duke of Urbino (1514-1574), With his Armor by Filippo Negroli",
6      "artist": "",
7      "date": "ca. 1580-85"
8  },
9  {
10     "object_id": "35968",
11     "object_number": null,
12     "title": "清 佚名 台南地區荷蘭城堡|Forts Zeelandia and Provintia and the City of Tainan",
13     "artist": "Unidentified artist",
14     "date": "19th century"
15  },
16 ]

```

Рисунок 3 – Результат подготовки.

2. Загрузка и обработка изображений. Эта команда загружает указанное количество случайных картин из коллекции, применяет к ним гамма-коррекцию и сохраняет результаты. Загрузка происходит асинхронно, а обработка параллельно.

```

Terminal Local x + v
(.venv) PS D:\py_projects\PythonProject1> metetl process --input data/to_download.json --output images --num 5
23:10:34 - metetl.cli - INFO - =====
23:10:34 - metetl.cli - INFO - Metropolitan Museum of Art ETL Pipeline
23:10:34 - metetl.cli - INFO - =====
23:10:34 - metetl.cli - INFO - Выполнение команды: process
23:10:37 - metetl.cli - INFO - Запуск загрузки 5 изображений
23:10:37 - metetl.cli - INFO - Результаты будут сохранены в: images
23:10:37 - metetl.images.async_downloader - INFO - AsyncImageDownloader инициализирован. CSV: data\MetObjects.csv, output: images, max_attempts: 10
23:10:37 - metetl.images.async_downloader - INFO - Начинается загрузка 5 изображений
23:10:41 - metetl.images.async_downloader - INFO - Найдено 9005 картин в CSV
23:10:41 - metetl.images.async_downloader - INFO - Загрузка 5 изображений из 9005 доступных (максимум попыток на каждое: 10)
23:10:41 - metetl.images.async_downloader - INFO - Загрузка изображения 1 началась (ID: 53198), попытка 1/10
23:10:41 - metetl.images.async_downloader - INFO - Загрузка изображения 2 началась (ID: 705076), попытка 1/10
23:10:41 - metetl.images.async_downloader - INFO - Загрузка изображения 3 началась (ID: 485245), попытка 1/10
23:10:41 - metetl.images.async_downloader - INFO - Загрузка изображения 4 началась (ID: 62739), попытка 1/10
23:10:41 - metetl.images.async_downloader - INFO - Загрузка изображения 5 началась (ID: 53194), попытка 1/10
23:10:41 - metetl.images.async_downloader - WARNING - Попытка 1 для изображения 3 провалена: нет URL изображения для ID 485245
23:10:41 - metetl.images.async_downloader - INFO - Загрузка изображения 3 началась (ID: 45418), попытка 2/10
23:10:42 - metetl.images.async_downloader - INFO - Загрузка изображения 1 завершена (ID: 53198) после 1 попыток
23:10:42 - metetl.images.async_downloader - INFO - Загрузка изображения 5 завершена (ID: 53194) после 1 попыток
23:10:42 - metetl.images.async_downloader - INFO - Загрузка изображения 3 завершена (ID: 45418) после 2 попыток
23:10:42 - metetl.images.async_downloader - INFO - Загрузка изображения 4 завершена (ID: 62739) после 1 попыток
23:10:42 - metetl.images.async_downloader - INFO - Загрузка изображения 2 завершена (ID: 705076) после 1 попыток
23:10:42 - metetl.images.async_downloader - INFO - Успешно загружено 5 из 5 изображений
23:10:42 - metetl.cli - INFO - Загружено изображений: 5

```

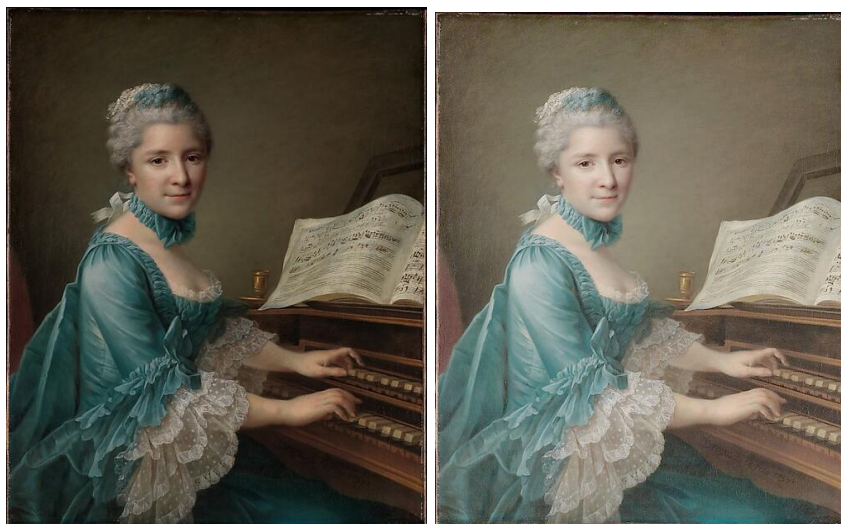
Рисунок 4 – Асинхронная загрузка изображений.

```

23:10:42 - metetl.images.parallel_processor - INFO - ParallelImageProcessor инициализирован. Output: images, workers: 4
23:10:42 - metetl.images.parallel_processor - INFO - Начинается параллельная обработка 5 изображений
23:10:42 - metetl.images.parallel_processor - INFO - Используемый метод обработки: gamma_correction
[PID 29388] Обработка 'gamma_correction' для изображения 3 началась
[PID 25936] Обработка 'gamma_correction' для изображения 2 началась
[PID 17816] Обработка 'gamma_correction' для изображения 4 началась
[PID 11440] Обработка 'gamma_correction' для изображения 1 началась
[PID 25936] Обработка 'gamma_correction' для изображения 2 завершена
[PID 29388] Обработка 'gamma_correction' для изображения 3 завершена
[PID 25936] Обработка 'gamma_correction' для изображения 5 началась
[PID 17816] Обработка 'gamma_correction' для изображения 4 завершена
[PID 11440] Обработка 'gamma_correction' для изображения 1 завершена
[PID 25936] Обработка 'gamma_correction' для изображения 5 завершена
23:10:43 - metetl.images.parallel_processor - INFO - Обработка завершена, сохранено 5 изображений
23:10:43 - metetl.images.parallel_processor - INFO - 1. ID: 53198 -> 1_53198_gamma_correction.png
23:10:43 - metetl.images.parallel_processor - INFO - 2. ID: 705076 -> 2_705076_gamma_correction.png
23:10:43 - metetl.images.parallel_processor - INFO - 3. ID: 45418 -> 3_45418_gamma_correction.png
23:10:43 - metetl.images.parallel_processor - INFO - 4. ID: 62739 -> 4_62739_gamma_correction.png
23:10:43 - metetl.images.parallel_processor - INFO - 5. ID: 53194 -> 5_53194_gamma_correction.png
23:10:43 - metetl.cli - INFO - Результаты обработки:
23:10:43 - metetl.cli - INFO - 1. ID: 53198 -> 1_53198_gamma_correction.png
23:10:43 - metetl.cli - INFO - 2. ID: 705076 -> 2_705076_gamma_correction.png
23:10:43 - metetl.cli - INFO - 3. ID: 45418 -> 3_45418_gamma_correction.png
23:10:43 - metetl.cli - INFO - 4. ID: 62739 -> 4_62739_gamma_correction.png
23:10:43 - metetl.cli - INFO - 5. ID: 53194 -> 5_53194_gamma_correction.png
23:10:43 - metetl.cli - INFO - Всего обработано: 5 изображений

```

Рисунок 5 – Параллельная обработка изображений.



Рисунки 6, 7 – Оригинал слева и обработка гамма-коррекцией справа.

3. Анализ данных и построение графиков. Эта команда запускает анализ CSV-файла, вычисляет статистику по культурам и строит графики (столбцовая диаграмма и временной график для культуры с самой длинной историей).

```

(.venv) PS D:\py_projects\PythonProject1> metetl analyze --csv data\MetObjects.csv --output-dir data\plots
23:23:53 - metetl.cli - INFO - =====
23:23:53 - metetl.cli - INFO - Metropolitan Museum of Art ETL Pipeline
23:23:53 - metetl.cli - INFO - =====
23:23:53 - metetl.cli - INFO - Выполнение команды: analyze
23:24:10 - metetl.cli - INFO - Запуск анализа данных из data\MetObjects.csv
23:24:10 - metetl.cli - INFO - Результаты будут сохранены в: data\plots
23:24:10 - metetl.cli - INFO - Чтение данных...
23:24:10 - metetl.analysis.generators - INFO - Начало аккумуляции статистик
23:24:14 - metetl.analysis.generators - INFO - Всего обработано записей: 413554
23:24:14 - metetl.analysis.generators - INFO - Уникальных культур: 6672
23:24:14 - metetl.analysis.generators - INFO - Аккумуляция завершена. Сформировано 20016 записей статистик
23:24:14 - metetl.cli - INFO - Вычисление статистик...
23:24:14 - metetl.cli - INFO - Top-10 культур: nan, American, French, Japan, China, Italian, Japanese, British, German, Roman
23:24:14 - metetl.analysis.analysis - WARNING - Пропущена культура: nan (нет в статистиках)
23:24:14 - metetl.analysis.plotting - INFO - Построение столбчатой диаграммы: data\plots\bar_chart.png
23:24:14 - metetl.analysis.plotting - INFO - Столбчатая диаграмма сохранена: data\plots\bar_chart.png
23:24:14 - metetl.analysis.analysis - INFO - Культура с самой длительной историей: American
23:24:14 - metetl.cli - INFO - Культура с самой длительной историей: American
23:24:14 - metetl.analysis.analysis - INFO - Сбор временных данных для культуры: American
23:24:17 - metetl.analysis.analysis - INFO - Собрано 149 точек данных для культуры American
23:24:17 - metetl.analysis.plotting - INFO - Построение временного графика для культуры American: data\plots\time_series_American.png
23:24:17 - metetl.analysis.plotting - INFO - Временной график сохранен: data\plots\time_series_American.png
23:24:17 - metetl.cli - INFO - Анализ завершен
23:24:17 - metetl.cli - INFO - Графики сохранены в: data\plots

```

Рисунок 8 – Вычисление статистик и построение графиков.

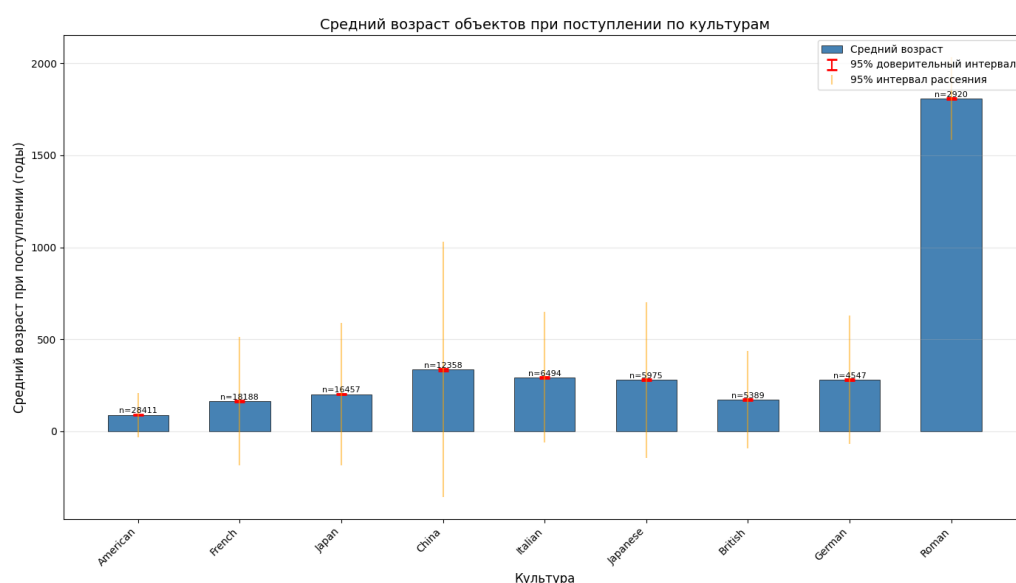


Рисунок 9 – График среднего возраста объектов.

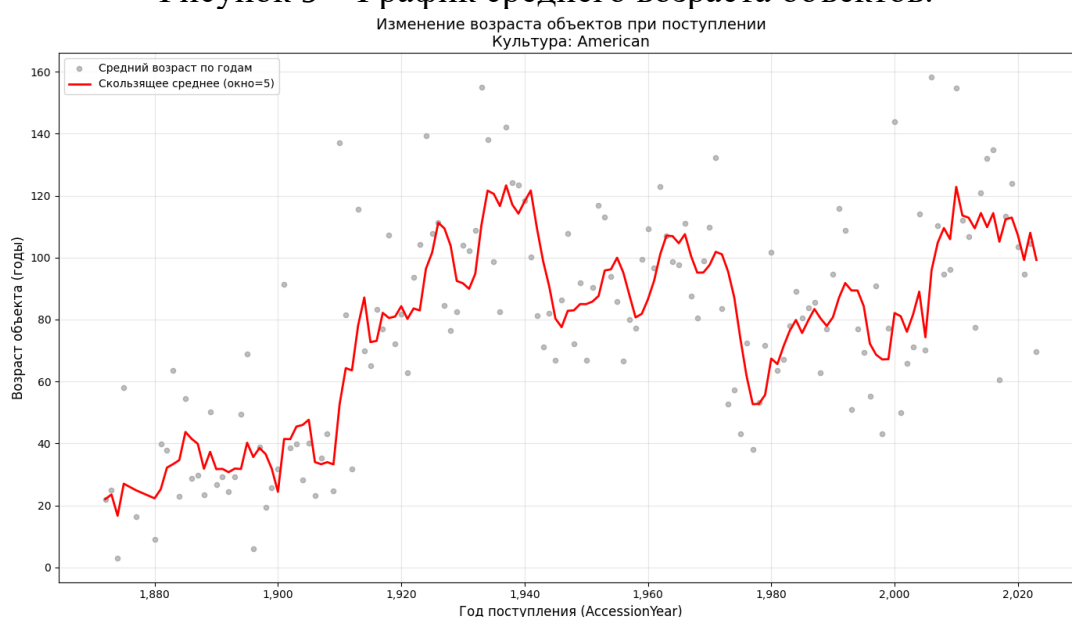


Рисунок 10 – График изменения возраста поступающих объектов.

ЗАКЛЮЧЕНИЕ

В результате выполнения полного цикла лабораторных работ был создан готовый к использованию и распространению Python-пакет. Проект охватывает спектр технологий и демонстрирует их практическое применение для решения реальной задачи по работе с открытыми данными.

Выполнены задачи:

- Разработана модульная и расширяемая архитектура, основанная на принципах ООП, которая обеспечивает четкое разделение ответственности между компонентами проекта.
- Интегрированы технологии асинхронного и параллельного программирования, что позволило оптимизировать узкие места приложения (сетевой ввод-вывод и вычислительно-емкие операции) и достичь значительного ускорения работы.
- Реализован эффективный пайплайн для анализа больших табличных данных с использованием генераторов и библиотеки `pandas`, что гарантирует масштабируемость программы.
- Создан удобный интерфейс командной строки, предоставляющий доступ ко всем функциям приложения.
- Приложение полностью подготовлено к сборке и распространению в виде Python-пакета.

Все цели, поставленные в рамках лабораторных работ, полностью выполнены. Полученный опыт и созданное приложение могут служить основой для дальнейших исследований в области обработки больших наборов мультимедийных данных.

ПРИЛОЖЕНИЕ А

Ссылки на репозиторий и пакет

Репозиторий на GitHub:

<https://github.com/Ziorde/6302balakirevio>

Пакет на TestPyPI:

<https://test.pypi.org/project/balakirev6302/>

ПРИЛОЖЕНИЕ Б

Список зависимостей

Список зависимостей (из pyproject.toml):

- Numpy – v. 1.20.0
- Opencv-python – v. 4.5.0,
- Pandas – v. 1.3.0,
- Matplotlib – v. 3.4.0,
- Scipy – v. 1.7.0,
- Requests – v. 2.25.0,
- Aiohttp – v. 3.8.0,
- Aiofiles – v. 0.8.0.

ПРИЛОЖЕНИЕ В

Код конфигурации логгера

```
{
  "version": 1,
  "disable_existing_loggers": false,
  "formatters": {
    "file_formatter": {
      "format": "%(asctime)s - %(name)s - %(levelname)s - %(filename)s:%(lineno)d - %(message)s",
      "datefmt": "%Y-%m-%d %H:%M:%S"
    },
    "console_formatter": {
      "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s",
      "datefmt": "%H:%M:%S"
    }
  },
  "handlers": {
    "file_handler": {
      "class": "logging.FileHandler",
      "level": "DEBUG",
      "formatter": "file_formatter",
      "filename": "logs/metetl.log",
      "encoding": "utf-8",
      "mode": "a"
    },
    "console_handler": {
      "class": "logging.StreamHandler",
      "level": "INFO",
      "formatter": "console_formatter",
      "stream": "ext://sys.stdout"
    }
  }
}
```

}

}

}